# MEMORIA TÉCNICA — EUSKADI PASS

**Proyecto Individual JS | Bootcamp Full Stack con IA | The Bridge**

Autora: Ainara Uranga | Fecha: 6 de Noviembre 2026

---

## 1. Descripción del proyecto

### 1.1. Contexto y justificación del Trabajo

Durante los últimos años Euskadi cuenta con una oferta cultural amplia/diversa y en constante crecimiento. Según el Observatorio Vasco de la Cultura, en 2025 se registraron 22.425 eventos culturales en Kulturklik, una media de 61,4 al día, mientras que en el segundo trimestre del 2026 esa cifra creció un 3,2% interanual, hasta los 5.949 eventos.

Ese logro es en parte gracias al trabajo del Departamento de Cultura y Política Lingüística del Gobierno Vasco gestionando la plataforma Kulturklik, la mayor base de datos pública y centralizada de la oferta cultural de Euskadi. Su misión institucional es visibilizar la red de eventos, espacios y espectáculos de la comunidad autónoma. Sin embargo, su formato de consulta tradicional actúa como un repositorio de información pasivo que presenta listas densas y genera fricciones reales:

- **No existe una app nativa unificada para Euskadi:** Kulturklik opera como web de consulta, sin funcionalidades de interacción, guardado ni personalización.
- **El consumo cultural tiende a ser provincial:** Sin una herramienta que ponga en valor el conjunto del territorio, el consumo interterritorial queda supeditado al azar o a la iniciativa individual.
- **No hay incentivo activo para explorar:** La agenda existe, pero no hay ningún mecanismo que motive al usuario a salir de su zona habitual o descubrir tipos de evento nuevos.

Euskadi Pass propone cubrir ese hueco: una app que digitaliza y gamifica la agenda de Kulturklik, incentiva la exploración de los tres territorios y convierte la asistencia a eventos en una experiencia acumulativa con recompensas. Inspirada en Gipuzkoa Passport y Botxo Passport, su objetivo es incitar al turismo cultural intraterritorial.

### 1.2. Objetivos de aprendizaje

El objetivo general es el desarrollo de un MVP mobile-first de catálogo cultural gamificado, consolidando fundamentos de desarrollo frontend mediante:

- **POO:** Modelado del dominio con la clase `Event` y métodos encapsulados para estructurar la lógica de negocio.
- **Consumo de APIs asíncronas:** Peticiones `fetch` con `async/await` para integrar datos dinámicos desde la API de Kulturklik.
- **Manipulación dinámica del DOM:** Interfaz interactiva Mobile-First que responde a filtrado, búsqueda, favoritos y simulación de QR sin recargar la página.

- **Persistencia local (localStorage):** Estado del pasaporte (sellos, favoritos) persistente entre sesiones.

### 1.3. Alcance

| En Alcance (MVP) | Fuera de Alcance |
|---|---|
| Pantalla de Inicio: misión activa(estática) y sección de eventos recomendados | Lectura de QR real (cámara móvil): queda para siguiente fase |
| Pantalla de Explorar: listado desde API, búsqueda por texto, filtros por provincia y categoría, favoritos con localStorage | Backend y base de datos real |
| Pantalla de Mi Pasaporte: colección de sellos y catálogo de recompensas simulado | Recompensas canjeables reales |
| Pantalla de Favoritos: listado filtrable de eventos guardados | Versión física del pasaporte |
| Pantalla de Detalle de evento | Onboarding de preferencias |
| Simulación de escaneo QR (modal) | Pantalla de mapa / agenda visual |
| Bloqueo de vista en escritorio (mobile-only) | Modo oscuro/claro y exportación de datos |

> ***Nota sobre el QR:*** *El planteamiento inicial contemplaba que el usuario mostrase su propio QR para que el personal lo validase. Se descartó en favor del modelo inverso —cada evento tiene su QR y el usuario lo escanea/simula— por ser técnicamente autónomo y no depender de infraestructura externa.*

### 1.4. Declaración sobre el uso de inteligencia artificial generativa

Se han empleado Claude (Anthropic) y Gemini (Google) como apoyo en: ideación y estructuración, consultas técnicas durante el desarrollo, detección y depuración de errores en el código, y apoyo en la documentación. Todo el código fuente ha sido comprendido, adaptado y ejecutado de forma manual y crítica por la autora.

---

## 2. Actores (público objetivo)

### Proto-persona 1 — Ane, 26 años, Bilbao

Consume cultura habitualmente en Bizkaia: conciertos, exposiciones, teatro ocasional. Conoce Kulturklik pero no la usa —le resulta densa y no le aporta nada que no pueda buscar en Instagram. No va a eventos en Gipuzkoa o Araba no porque no quiera, sino porque simplemente no se le ocurre. Le motiva la gamificación, las recompensas y el factor "coleccionar". Usa el móvil para todo.

**Necesidad principal:** descubrir oferta cultural fuera de su entorno habitual sin tener que buscarla activamente.

### Proto-persona 2 — Mikel, 48 años, Donostia

Consume cultura de forma ocasional, principalmente en Gipuzkoa. Tiende a repetir los mismos espacios. Le engancha más la idea de descubrir sitios nuevos y acumular logros que la gamificación pura. Una recompensa tangible sería el empujón que necesita para probar algo fuera de su zona.

**Necesidad principal:** motivación externa para explorar la oferta cultural de otros territorios.

---

## 3. Requisitos

### 3.1. Requisitos funcionales

- **RF-01 (Catálogo y Filtrado):** Consumir la API de Kulturklik y listar eventos filtrables dinámicamente por provincia (Bizkaia, Gipuzkoa, Araba) y categoría.
- **RF-02 (Búsqueda):** Búsqueda por texto en tiempo real sobre nombre, descripción y municipio.
- **RF-03 (Gamificación y Sellos -visual):** La pantalla Mi Pasaporte muestra una colección de sellos provinciales y un modal de simulación de QR. La lógica de desbloqueo dinámico queda planificada para la siguiente fase.
- **RF-04 (Sistema de Recompensas -visual):** El catálogo de recompensas está implementado como contenido estático visual. La lógica de puntos y canje queda planificada para la siguiente fase.
- **RF-05 (Favoritos):** Guardar y gestionar eventos en favoritos persistidos en localStorage.

### 3.2. Requisitos no funcionales

- **RNF-01 (Mobile-First):** Interfaz diseñada para móvil con barra de navegación inferior. Vista bloqueada en escritorio.
- **RNF-02 (Rendimiento Asíncrono):** Peticiones a la API con `async/await` y gestión de errores mediante `try/catch`.
- **RNF-03 (Persistencia de Sesión):** Estado del usuario persistente entre sesiones mediante localStorage.
- **RNF-04 (Mantenibilidad POO):** Código JS estructurado con clases y separación de responsabilidades por capas.

### 3.3. Historias de usuario

- **HU-01 (Exploración):** "Como usuario joven, quiero filtrar eventos por provincia y categoría para encontrar planes fuera de mi ciudad sin esfuerzo."
- **HU-02 (Gamificación):** "Como usuario motivado por el coleccionismo, quiero ver mis sellos provinciales desbloqueados para sentir un incentivo al viajar por Euskadi."
- **HU-03 (Validación):** "Como usuario asistiendo a un evento, quiero simular el escaneo de un código QR para registrar mi visita de forma instantánea."

---

## 4. Reglas de negocio

- **RN-01 (Sellos):** Cada simulación de QR exitosa otorga un sello correspondiente a la provincia del evento. Implementación de desbloqueo dinámico pendiente para siguiente fase
- **RN-02 (Misiones — próxima fase):** Las misiones evaluarán automáticamente criterios de progreso (ej. 1 evento en cada provincia) para otorgar insignias. Diseñadas en el modelo, implementación pendiente.
- **RN-03 (Recompensas simuladas):** El catálogo de recompensas está implementado como contenido estático visual, pendiente de lógica de canje real.

---

## 5. Datos, servicio externo y mapeo

### 5.1. API

Endpoint público de Kulturklik a través de Open Data Euskadi:

`https://api.euskadi.eus/culture/events/v1.0/events/upcoming`

Las peticiones se realizan con `fetch` y `async/await` desde `src/core/Event.js`. Se soporta paginación mediante los parámetros `limit` y `page`.

### 5.2. Datos complementarios

Los sellos y recompensas son contenido estático gestionado en el HTML/CSS. El estado de favoritos persiste en localStorage.

### 5.3. Mapeo JSON → clase

Cada ítem de la respuesta de la API se instancia como un objeto de la clase `Event`, que encapsula la lógica de limpieza y normalización del dato crudo:

- Limpieza de etiquetas HTML en textos (`_cleanHtml`)
- Extracción de imagen con fallback a placeholder (`_extractImage`)
- Conversión de código de provincia NORA a texto (`_calculateProvince`): `"48"` → Bizkaia, `"20"` → Gipuzkoa, `"1"` → Araba
- Normalización de fecha sin hora (`_cleanDate`)

---

## 6. Diseño del modelo y de la interfaz

### 6.1. UML de clases

| Clase | Responsabilidad | Relación |
|---|---|---|
| `Event` | Clase principal. Propiedades: `id`, `name`, `imageUrl`, `type`, `description`, `province`, `municipality`, `startDate`, `source`. Métodos privados de limpieza del dato crudo de la API. | Base del dominio |

| Clase | Responsabilidad | Relación |
|---|---|---|
| `FetchError` | Extiende `Error`. Captura errores HTTP de la API con `status` y mensaje personalizado. | Independiente |

> ***Nota de diseño:*** *El modelo inicial contemplaba clases adicionales (`Usuario`, `Mision`, `MisionManager`) para encapsular el estado del usuario y el sistema de misiones. En el MVP, el estado de favoritos se gestiona directamente en `localStorage` a través del módulo `localStorage.js`, y las misiones son contenido estático. Estas clases están planificadas para la siguiente fase.*

### 6.2. Diagrama de flujo de navegación

1. **Home (`index.html`):** Misión activa (estática) y listado de eventos recomendados desde la API.
2. **Explorar (`explore.html`):** Listado completo con buscador, filtro de provincia y filtros chip por categoría.
3. **Detalle (`detail.html`):** Vista individual del evento, accesible desde cualquier tarjeta.
4. **Favoritos (`favorites.html`):** Listado filtrable de eventos guardados en localStorage.
5. **Mi Pasaporte (`pass.html`):** Sellos provinciales, catálogo de recompensas y modal de simulación de QR.
6. **Perfil (`profile.html`):** Datos de sesión simulada.

### 6.3. Organización de responsabilidades

| Capa | Archivos | Responsabilidad |
|---|---|---|
| Dominio | `src/core/Event.js` | Clase `Event`: modelo de dato y mapeo desde la API |
| Servicios | `src/core/services/api.js`, `src/core/localStorage.js` | Conexión a la API y gestión de persistencia en localStorage |
| Utilidades | `src/core/utils/filters.js`, `src/core/utils/errors.js` | Lógica de filtrado combinado y clase `FetchError` |
| Presentación | `src/core/ui/renderer.js` | Renderizado dinámico de tarjetas y botones de filtro |
| Control | `src/core/index.js` | Orquestación: inicialización, paginación y coordinación de filtros |
| Componentes | `src/core/components/detail.js`, `src/core/components/pass.js` | Lógica específica de las vistas de detalle y pasaporte |

---

## 7. Estructura y recorrido de la aplicación

### 7.1. Archivos principales

```
├── index.html
└── src/
```

```
├── html/
│   ├── explore.html
│   ├── detail.html
│   ├── favorites.html
│   ├── pass.html
│   └── profile.html
├── css/
│   ├── generalStyles.css
│   ├── indexStyles.css
│   ├── exploreStyles.css
│   ├── detailStyles.css
│   ├── pass.css
│   └── profile.css
├── core/
│   ├── index.js
│   ├── Event.js
│   ├── Favorites.js
│   ├── localStorage.js
│   ├── services/
│   │   └── api.js
│   ├── ui/
│   │   └── renderer.js
│   ├── utils/
│   │   ├── filters.js
│   │   └── errors.js
│   └── components/
│       ├── detail.js
│       └── pass.js
└── assets/
```

### 7.2. index.html

Página de inicio. Contiene la barra de navegación inferior, la sección de misión activa (estática) y el contenedor de eventos recomendados, que se puebla dinámicamente desde la API al cargar.

### 7.3. Vistas adicionales

La aplicación es **multi-página HTML** con módulos JS independientes por vista. La navegación entre pantallas se realiza mediante la barra inferior (links HTML estándar). El DOM de cada vista se manipula dinámicamente desde su módulo JS correspondiente sin recargar la página completa.

---

## 8. Acceso a datos, estado y limitaciones

### 8.1. Persistencia

El estado de favoritos se persiste en localStorage bajo la clave `"favorites"` como array de objetos `Event` serializado en JSON. Las operaciones disponibles son: `toggleFavorite`, `addToFavorites`, `removeFavorites`, `isInFavorites` y `clearFavorites`, todas en `src/core/localStorage.js`.

### 8.2. Gestión de fallos del servicio externo

Las peticiones a la API están envueltas en bloques `try/catch` en `Event.js`. Si la API no responde o devuelve un error HTTP, se captura una instancia de `FetchError` (definida en `utils/errors.js`) y se devuelve `null` al llamador, que muestra un mensaje de error al usuario sin romper la interfaz.

### 8.3. Restricciones observadas

- Dependencia de la disponibilidad y estructura pública del endpoint de Open Data Euskadi.
- El localStorage puede perderse si el usuario limpia la caché del navegador.
- El detalle del evento (`detail.html`) lee el dato desde `localStorage` (clave `"selectedEvent"`). Si el usuario accede directamente a la URL sin pasar por el listado, la vista no renderiza contenido.

---

## 9. Pruebas y comprobaciones

- Validación del consumo asíncrono y mapeo de objetos en consola del navegador.
- Comprobación de la persistencia de favoritos tras recargar la página.
- Verificación del comportamiento Mobile-First y respuesta de los filtros combinados (provincia + categoría + texto).
- Comprobación del flujo de toggle de favoritos (añadir, eliminar, estado visual sincronizado).

---

## 10. Decisiones técnicas

- **POO en Vanilla JS:** Permite cumplir los requisitos académicos sin la sobrecarga de un framework para un MVP de entrega rápida.
- **Multi-página HTML en lugar de SPA:** Simplifica la gestión del estado y el enrutamiento para el alcance del proyecto, con módulos JS independientes por vista.
- **Modelo inverso de validación QR:** El usuario escanea/simula el QR del evento, no al revés, eliminando la dependencia de infraestructura externa en taquilla.
- **Open Data Euskadi como fuente de datos:** Fuente oficial consolidada (22.425 eventos anuales) que aporta realismo y utilidad pública al proyecto.
- **Filtros combinados con estado compartido:** La función `applyFilters` en `filters.js` centraliza la lógica de provincia + categoría + texto en una sola pasada, evitando filtros independientes que no se combinen correctamente.

---

## 11. Posibles ampliaciones y conclusiones

### 11.1. Mejoras priorizadas

1. Implementar las clases `Usuario`, `Mision` y `MisionManager` para encapsular el estado del usuario y activar el sistema de misiones dinámicas.

2. Añadir fallback en `detail.html`: si no hay dato en localStorage, hacer fetch a la API con el `id` de la URL.
3. Backend ligero con base de datos real para autenticación y sincronización multidispositivo.
4. Integración de la cámara nativa para lectura real de QR en puntos culturales físicos.
5. Alianzas con oficinas de turismo locales para habilitar el canje real de recompensas.

### 11.2. Conclusión

Euskadi Pass demuestra cómo una capa de diseño UX/UI y mecánicas de gamificación pueden transformar un repositorio de datos públicos pasivo en una herramienta atractiva para incentivar el turismo cultural intraterritorial en Euskadi. El proyecto aplica con rigor los fundamentos de POO, consumo de APIs asíncronas, manipulación del DOM y persistencia local, sentando una base técnica sólida para su posterior escalado.